

INTER IIT TECH MEET 15.0 *PREPATHON PS*

QUANTITATIVE FINANCE

DCs: Tarun Agarwal (6393805918), Lakshya Vats (7015873839)

Quantitative Finance — Inter IIT Prepathon

Multi-Alpha Research Lab

Research, validate and combine multiple sources of systematic alpha.

This document defines the research problem, tasks, evaluation criteria and deliverables. Required interfaces, rules and the signal library reference are in the companion *Technical Documentation*.

1. PROBLEM STATEMENT

Background. A quantitative trading system is not built by finding one profitable rule. In practice, researchers must turn raw market information into testable **signals**, **determine whether an observed return is genuine or a consequence of noise and overfitting**, **understand whether different strategies are actually bringing different sources of alpha**, and finally decide how capital should be allocated among them.

You are given historical market data together with a set of **anonymised signals** drawn from a supplied signal library (predominantly boolean state flags, with a small number of bounded continuous or ordinal signals) spanning price-based, band-based and volume-based categories. **Each anonymised signal corresponds to one signal in the supplied signal library. The identities of the signals are hidden in the competition data, but their definitions and intended interpretation are available through the supplied signal library.**

Your task is to build a research system that moves from **data → strategy → evidence → distinct alpha → portfolio**. The problem is cumulative: later tasks must use the outputs of earlier tasks.

1. **Task 1: Data & Backtesting Infrastructure**
2. **Task 2: Alpha Discovery & Research**
3. **Task 3: Alpha Portfolio Construction & Dynamic Allocation**

2. TASK 1 — DATA & BACKTESTING INFRASTRUCTURE

Objective

Build a reusable quantitative research and backtesting framework in which data handling, strategy logic, execution, portfolio accounting and performance analysis are independent components.

The framework must support the development and comparison of multiple strategies under identical execution and accounting assumptions. A strategy should be replaceable without rewriting the backtester, execution engine or portfolio logic.

A **baseline strategy** must be included and the complete pipeline must run end-to-end before proceeding to Task 2. (See the Technical Documentation for the required project structure, classes/functions and research-integrity requirements.)

3. TASK 2 — ALPHA DISCOVERY & RESEARCH

Objective

Use the supplied signal library to develop multiple trading hypotheses, test whether they contain robust predictive information, and determine which resulting strategies represent genuinely distinct sources of alpha.

3.1 Research Setting

The supplied signal library is the **predictive information set** for this task. Participants may combine, transform or condition on the supplied signals in any way, but **no other predictive information may be introduced**.

Raw price and volume series remain available, but strictly in two roles:

- as the source of the **forward-return target** used for correlation research and for fitting/training any model (i.e. as a supervised-learning label, not a feature); and
- for **trade execution and portfolio accounting** inside the backtester.

Price and volume must never appear as an input to `generate_features` or `generate_signal`. A strategy's trading decision must be fully determined by the supplied signal library (and any model fitted on it) — never by price or volume directly. The precise timing convention (which candle's signal values may be used, and which candle the resulting trade executes on) and the mandated transaction cost are given in the Technical Documentation's Execution Assumptions.

The purpose is not to maximize the number of strategies. A strategy should be supported by a clear hypothesis, a reproducible construction and evidence that its observed performance is not merely an in-sample artifact.

Given the constrained signal library, teams must develop at least **three candidate strategies**, and may develop up to around **six** where each represents a genuinely distinct hypothesis. Strategies should represent meaningfully different hypotheses rather than multiple parameter sweeps of the same rule — a smaller set of genuinely independent ideas is preferred over a larger set that pads toward a target count.

3.2 Required Research Questions

For each candidate strategy, investigate as appropriate:

- What market behaviour or hypothesis motivates the strategy?
- Which supplied signals provide the information used by the strategy?
- What horizon and trading rule are being tested?
- Is the observed alpha statistically distinguishable from noise?
- Does the effect persist across different time periods or market conditions?
- How sensitive is the strategy to transaction costs, turnover and reasonable parameter changes?
- Where does the strategy fail, and are those failures systematic?

The final research must then address a second question: **are the strategies actually different sources of alpha, or are several strategies repackaging the same return behaviour?**

3.3 Independent Alpha Analysis

Let the common chronological observations for N strategies be $t = 1, \dots, T$, where all strategy return vectors are defined. For strategy i ,

$$\mathbf{r}_i = [r_{i(1)}, \dots, r_{i(T)}]^T \in \mathbb{R}^T$$

Construct

$$\mathbf{R} = [\mathbf{r}_1 \mathbf{r}_2 \dots \mathbf{r}_N] \in \mathbb{R}^{T \times N}$$

Treat the strategy return streams as vectors and study the linear structure of their span. A suitable orthogonalization method, such as QR decomposition with column pivoting or Gram–Schmidt, may be used to determine:

- the effective dimensionality of the strategy-return space;
- which strategies contribute new directions to the span;
- which strategies are largely explained by previously identified directions;
- the residual contribution of a strategy after projection onto an existing alpha subspace; and
- whether the identified independent directions remain stable through time.

Correlation alone is not sufficient for this analysis. The objective is to study incremental linear information in the return space, not merely pairwise correlation.

A mathematically distinct return direction is not, by itself, proof of economic independence. Teams must therefore interpret the linear-algebraic results together with the statistical and robustness evidence.

3.4 Strategy Selection

The output of Task 2 should be a defensible final strategy set. Selection should consider, jointly:

- magnitude and consistency of alpha;
- statistical evidence;
- robustness across time/conditions;
- implementation costs and turnover; and
- incremental information relative to the other selected strategies.

4. TASK 3 — ALPHA PORTFOLIO CONSTRUCTION & DYNAMIC ALLOCATION

Objective

Given the strategy set selected from Task 2, determine how capital should be allocated across strategies and whether a dynamic allocation process can improve the portfolio without simply increasing risk or trading activity.

A portfolio manager is not interested only in which strategy has the highest standalone return. The allocation decision must consider expected alpha, systematic exposure, residual risk, interactions between strategies, concentration, turnover and the cost of changing positions.

4.1 Required Research

The final system must investigate:

- strategy-specific alpha and systematic exposure;
- portfolio volatility, drawdown and concentration;
- diversification and interactions among strategy returns;
- turnover and transaction-cost impact;
- static versus dynamic allocation;
- whether strategy effectiveness changes over time in a way that can be exploited without using future information; and
- whether the learned component's contribution is genuine given the limited effective sample size, rather than an artefact of overfitting.

A factor model may be used to separate systematic and strategy-specific components. For example,

$$r_{i(t)} = \alpha_i + \beta_i^T F(t) + \varepsilon_{i(t)}$$

where $F(t)$ denotes the chosen systematic factors. The exact factor specification is part of the team's research and must be justified.

For static optimisation, teams may formulate an objective balancing expected alpha against portfolio risk and other constraints. A valid formulation should make its assumptions explicit rather than optimizing a single historical return metric without risk control.

For dynamic allocation, the system should use only information available at the allocation time. The required learned component should estimate strategy effectiveness, expected contribution, or another allocation-relevant quantity; it should not simply be a generic next-price predictor disconnected from the portfolio decision. Because the training history for this component is short, its value must be established rigorously rather than assumed — see the Model Discipline requirements in the Technical Documentation.

4.2 Required Allocation Comparisons

The final portfolio must be compared against the following common baselines:

1. best individual strategy;
2. equal-weight allocation;
3. risk-based allocation;
4. optimisation-based allocation; and

5. proposed dynamic allocation.

The comparison must use the same market data, execution assumptions, cost model and evaluation period for all methods.

5. EVALUATION & RANKING

The competition is designed to reward research that generalises, not a backtest that is optimized for one historical sample. Evaluation will therefore separate **research/development data** from **unseen evaluation data**.

5.1 Checkpoint Evaluation

- **Task 1:** implementation correctness, modularity, backtesting integrity and reproducibility.
- **Task 2:** quality of hypotheses, empirical evidence, statistical validity, robustness, cost sensitivity and distinctness of alpha.
- **Task 3:** quality of portfolio construction, risk control, allocation methodology, out-of-sample generalisation, and whether the learned component's contribution survives the required model-discipline checks (capacity justification, walk-forward validation, null-baseline comparison).

5.2 Final Portfolio Evaluation

All submitted strategies and allocation methods will be evaluated under common execution and cost assumptions on the designated evaluation data. Final evaluation will consider:

- net return and annualized return;
- Sharpe and/or other risk-adjusted performance;
- maximum drawdown and downside risk;
- turnover and transaction-cost impact;
- stability across evaluation periods; and
- improvement relative to the required allocation baselines.

In-sample performance alone will not determine the final ranking. A method that achieves a high development-set return by overfitting but fails on unseen data is expected to rank below a more robust method.

The organisers may use additional stress or robustness checks, including changes to reasonable execution assumptions or evaluation sub-periods, to identify unstable solutions.

6. DELIVERABLES

6.1 Task 1

- Complete modular codebase implementing the required classes and interfaces.
- Working end-to-end data and backtesting pipeline.
- Baseline strategy and backtest output.
- Short implementation note describing design decisions and research-integrity checks.

6.2 Task 2

- At least three (up to around six) genuinely distinct candidate strategies and their code.
- Research report covering hypotheses, signal usage, construction, results and failure modes.
- Statistical and robustness analysis.
- Independent-alpha / return-space analysis.
- Final selected strategy set with justification.

6.3 Task 3

- Portfolio construction and allocation code.
- Comparison against all required allocation baselines.
- Factor/risk analysis and dynamic-allocation research.
- Model-discipline evidence for the learned component: capacity justification, walk-forward fold results and null-baseline comparison.

- Final evaluation on development and unseen data, where applicable.
- Final research report explaining the allocation methodology and evidence for improvement.

6.4 Submission Package

Submit a single .zip containing the complete codebase, configuration, reproducible results, reports and any required supporting files. The project must be runnable without manual modification of internal paths.

7. SCOPE AND ASSUMPTIONS

The problem statement defines the common data, signal information, evaluation conventions and required interfaces. Participants are free to choose the mathematical/statistical methods used inside those interfaces unless a method is explicitly mandated.

Where a modelling choice is not specified, make a reasonable assumption, apply it consistently and document it. Undocumented changes to the supplied data, signal availability, execution rules or evaluation procedure will be treated as errors rather than assumptions.

8. SUBMISSION DEADLINES

Deadlines

Each checkpoint is submitted and evaluated independently, on the schedule below.

Checkpoint	Deadline
Task 1 — Data & Backtesting Infrastructure	14 September 2026 (Monday)
Task 2 — Alpha Discovery & Research	18 September 2026 (Friday)
Task 3 — Alpha Portfolio Construction & Dynamic Allocation	21 September 2026 (Monday)

Quantitative Finance — Technical Documentation

Multi-Alpha Research Lab — Technical Documentation

Required interfaces, rules and the signal library reference.

Companion to the *Problem Statement* document, which defines the research problem, tasks, evaluation criteria and deliverables.

1. TASK 1 — DATA & BACKTESTING INFRASTRUCTURE

1.1 Research Integrity Requirements

The framework must explicitly account for:

- chronological ordering and data-quality issues;
- missing observations and duplicate observations;
- signal availability at the time a decision is made;
- transaction costs and slippage;
- position and cash/P&L accounting;
- performance measurement from the resulting equity/return series; and
- prevention or detection of look-ahead information.

1.2 Execution Assumptions

These are fixed and mandatory — not free parameters for teams to choose.

Transaction cost. Apply **0.05% per side** (0.05% of trade notional on entry, 0.05% on exit, so a full round trip costs 0.10%). This applies uniformly to every executed trade in every task; do not substitute a different assumption.

Timing convention. A trading decision for candle t may use signal-library values available only **through candle $t - 1$'s close** (i.e. the most recently completed candle at the time the decision is made). The resulting trade must be executed at **candle t 's open** price. Signal values from candle t itself (or later) must never enter the decision for candle t .

Concretely, the per-candle loop is:

```
for t in candles:
    inputs_t = signal_library rows up to and including t-1
    decision_t = strategy.generate_signal(inputs_t)
    execute decision_t at price = open[t]
```

`generate_signals` is expected to enforce the $t - 1$ cutoff before calling `generate_signal`, and `execute_signals / execute` are expected to fill at the corresponding candle's open — never at that candle's close or any other price.

1.3 Required Project Structure

```
project/
├─ main.py
├─ config.py
├─ data_loader.py
├─ data_cleaner.py
├─ feature_engine.py
├─ strategy.py
├─ execution_engine.py
├─ portfolio.py
├─ performance.py
├─ backtester.py
├─ strategies/
└─ baseline_strategy.py
```


Additional files may be added where required by the implementation.

1.4 Required Classes and Functions

The following interfaces are mandatory; internal implementation is open.

DataLoader

Method	Description
load(source)	Load supplied data.
validate_schema(data)	Validate the expected structure.

DataCleaner

Method	Description
validate(data)	Check data quality.
clean(data)	Return research-ready data.
sort_chronologically(data)	Enforce time order.
handle_missing(data)	Handle missing observations.
remove_duplicates(data)	Handle duplicate observations.
get_report()	Return data-quality diagnostics.

FeatureEngine

Method	Description
add_feature(data, name, function)	Register a derived feature.
transform(data)	Apply configured transformations.
compute_returns(data)	Construct return series.
rolling_feature(data, window, function)	Construct a rolling feature.
validate_no_lookahead(data)	Check information availability.

BaseStrategy

Method	Description
generate_features(data)	Define strategy inputs.
generate_signal(data)	Generate trading decisions.
fit(data)	Fit parameters/models where applicable.
predict(data)	Produce strategy outputs.
get_metadata()	Return strategy metadata.

ExecutionEngine

Method	Description
execute(signal, market_data)	Simulate execution.
apply_transaction_cost(trades)	Apply trading costs.
apply_slippage(trades)	Apply execution slippage.
record_trade(trade)	Record an executed trade.
get_trade_log()	Return trade history.

Portfolio

Method	Description
update(executions)	Update portfolio state.
mark_to_market(market_data)	Mark positions to market.
get_equity_curve()	Return portfolio value history.
get_positions()	Return position history.
get_pnl()	Return P&L series.

PerformanceAnalyzer

Method	Description
total_return()	Total return.
annualized_return()	Annualized return.
volatility()	Return volatility.
sharpe_ratio()	Sharpe ratio.
sortino_ratio()	Sortino ratio.
max_drawdown()	Maximum drawdown.
drawdown_duration()	Drawdown duration.
calmar_ratio()	Calmar ratio.
trade_statistics()	Trade statistics.
report()	Combined performance report.

Backtester

Method	Description
run(data, strategy)	Run a historical simulation.
generate_signals(data, strategy)	Generate strategy decisions.
execute_signals(signals, data)	Simulate signal execution.
update_portfolio(executions, data)	Update portfolio.
analyze(portfolio)	Analyze performance.
get_results()	Return backtest results.

2. TASK 2 — ALPHA DISCOVERY & RESEARCH

2.1 Required Research Modules

```
alpha_research.py    → AlphaResearch
robustness.py        → RobustnessTester
statistics.py        → StatisticalTester
orthogonality.py     → OrthogonalityAnalyzer

strategies/
├─ alpha_01.py
├─ alpha_02.py
└-- alpha_03.py    ( ... up to alpha_06.py, one file per strategy)
```

3. TASK 3 — ALPHA PORTFOLIO CONSTRUCTION & DYNAMIC ALLOCATION

3.1 Model Discipline for the Learned Component

The history available for training the learned allocation component is short, so its reported value must be established rigorously rather than assumed:

- **Capacity.** Model complexity must be justified relative to the effective sample size. Prefer simple, regularised or interpretable model classes; a high-capacity or black-box model requires explicit justification for why it is warranted at this data scale.
- **Validation.** Evaluate the learned component using rolling-window (walk-forward) refitting rather than a single train/validation/test split, so the reported result reflects multiple pseudo-out-of-sample folds rather than one.
- **Null baseline.** The learned component's contribution must be compared against a null baseline — for example, the same allocator trained on randomly permuted strategy-effectiveness labels, or a randomised allocation of equivalent turnover. An improvement that does not clearly clear this null baseline must be reported as statistically indistinguishable from noise, not presented as genuine signal.
- **Reporting.** The final report must state the train/validation/test performance gap for the learned component and discuss explicitly what would have to be true for the observed improvement to be an overfitting artefact rather than real.

A team that reports a smaller but null-baseline-validated improvement is expected to score higher than a team that reports a larger, unvalidated one.

3.2 Required Portfolio Modules

```
factor_model.py      → FactorModel
portfolio_optimizer.py → PortfolioOptimizer
meta_model.py        → AlphaMetaModel
dynamic_allocator.py → DynamicAllocator
final_evaluation.py  → FinalEvaluator
```

3.3 Required Classes and Functions

FactorModel

Method	Description
<code>fit(strategy_returns, factors)</code>	Estimate the factor representation.
<code>predict(strategy_returns, factors)</code>	Produce model outputs.
<code>get_alpha()</code>	Return strategy-specific alpha estimates.
<code>get_beta()</code>	Return systematic exposures.
<code>get_residuals()</code>	Return residual components.
<code>report()</code>	Return factor-model diagnostics.

PortfolioOptimizer

Method	Description
<code>fit(strategy_returns, risk_model, constraints)</code>	Configure the allocation problem.
<code>optimize()</code>	Determine portfolio weights.
<code>evaluate(weights)</code>	Evaluate a candidate allocation.
<code>get_weights()</code>	Return resulting weights.
<code>report()</code>	Return optimisation diagnostics.

AlphaMetaModel

Method	Description
<code>fit(strategy_history, market_state)</code>	Fit the allocation-relevant effectiveness model.
<code>predict(strategy_history, market_state)</code>	Estimate future strategy effectiveness.
<code>score(strategies)</code>	Produce strategy-level assessments.
<code>get_feature_importance()</code>	Return model-importance information where supported.
<code>report()</code>	Return model diagnostics, including capacity justification, walk-forward fold results and the required null-baseline comparison.

DynamicAllocator

Method	Description
<code>allocate(strategy_scores, constraints)</code>	Convert assessments into target weights.
<code>rebalance(current_weights, target_weights)</code>	Determine portfolio changes.
<code>get_allocation_history()</code>	Return allocation history.
<code>report()</code>	Return allocation diagnostics.

FinalEvaluator

Method	Description
<code>evaluate(strategy_set)</code>	Evaluate a strategy or portfolio method.
<code>compare(methods)</code>	Compare required allocation methods.
<code>robustness_test(results)</code>	Test stability of final results.
<code>report()</code>	Return final evaluation report.

4. GENERAL REQUIREMENTS

- All research and trading decisions must respect chronological information availability.
- No future observations may enter feature construction, model fitting, strategy decisions, allocation decisions or evaluation.
- Transaction costs must be incorporated using the fixed 0.05%-per-side rule in Execution Assumptions; slippage must be incorporated consistently and its modelling choice justified.
- Every trading decision must follow the timing convention in Execution Assumptions: signal values only through candle $t - 1$, execution at candle t 's open.
- Task 2 predictive inputs are strictly the supplied anonymised signal library; price and volume may be used only as a forward-return training target and for execution/accounting, never as a signal-generation input.
- Strategy and portfolio comparisons must use a common evaluation framework.
- Development results and unseen evaluation results must be clearly distinguished.
- Any parameter search or model selection must be performed using only information available in the corresponding development period.
- The code must be modular, reproducible and executable from the submitted project structure.
- Any assumptions made beyond the explicitly supplied rules must be stated clearly in the report.

5. SIGNAL LIBRARY

The supplied signal library contains **20 anonymised signals**, released as pre-computed columns aligned to the trading-date index, split across three categories by naming prefix: **PB0x (price-based)**, **BB0x (band-based)** and **VB0x (volume-based)**. Signals are predominantly boolean (0/1) state flags, with a small number of bounded

continuous values. All signals are causal — each is computed only from information available at or before its date, so no additional lookahead handling is required to use them directly.

Exact formulas and parameters are intentionally withheld; the tables below describe only what each signal broadly signifies. Recovering or validating the precise relationship between a signal and future returns is part of the Task 2 research, not something to be looked up.

Per the General Requirements, these 20 columns (and any combination or transformation of them) are the only permitted predictive inputs for Task 2 and Task 3 strategy and model logic. Raw price and volume remain available only as the forward-return training target and for execution/accounting.

5.1 Price-Based (PB)

Code	Type	Broadly signifies
PB01	Boolean	A short-term trend-direction flag.
PB02	Boolean	A longer-term trend-direction flag.
PB03	Boolean	Whether price is sitting above a short-term average reference level.
PB04	Boolean	Whether price is sitting above a longer-term average reference level.
PB05	Boolean	The sign of recent short-horizon price momentum.
PB06	Boolean	A recent price breakout to a local high.
PB07	Continuous	A normalised measure of price's distance from a medium-term trend reference.
PB08	Continuous	A normalised measure of trend strength / separation between two trend references.

5.2 Band-Based (BB)

Code	Type	Broadly signifies
BB01	Boolean	A breakout above an upper volatility band.
BB02	Boolean	A breakout below a lower volatility band.
BB03	Boolean	An overbought condition from a bounded momentum oscillator.
BB04	Boolean	An oversold condition from a bounded momentum oscillator.
BB05	Boolean	A period of unusually low volatility (a "squeeze").
BB06	Continuous	A normalised measure of where price sits within its recent volatility band.
BB07	Continuous	A normalised measure of the recent volatility level.

5.3 Volume-Based (VB)

Code	Type	Broadly signifies
VB01	Boolean	Above-average trading participation.
VB02	Boolean	A rising trend in cumulative volume flow.
VB03	Boolean	A price move unusually well supported by volume.
VB04	Boolean	A secondary participation-related flag.
VB05	Continuous	A normalised measure of how unusual current participation is relative to its recent history.

5.4 Dataset Link

The following link provides the anonymised signal library and historical market data for the competition:

https://drive.google.com/drive/folders/1mhkCSTsNEu-8wp_jNfU1N-AjWVSSVsGB?usp=sharing